Comparative Analysis and Performance Evaluation of a Two-Wheeled Self-Balancing Robot Using P, PI, and PID Controllers Tuned By Metaheuristic Algorithms

Md. Arman Hassan*, Kazi Md. Zareer, Mostafijur Rahaman, S. M. Hasibur Rahman, Tanzimur Rahman, Sumon Saha

[a]*Department of Mechanical Engineering, Bangladesh University of Engineering and Technology, Dhaka, 1000, Bangladesh*

Abstract

Two-wheeled self-balancing robots (TWSBRs) are unstable systems that resemble inverted pendulums. They need complex control strategies for staying balanced. Their small size and ability to move make them suitable for robotics research and autonomous mobility. Many studies have suggested control solutions using detailed modeling methods and nonlinear controllers. However, classical controllers still need more exploration, especially when tuned thoughtfully. In this study, we first used a mathematical modeling approach based on Mahler et al.'s work. We applied Euler–Lagrange equations to derive the nonlinear dynamics of the system, including gravitational, inertial, and motor-induced torques. We modeled friction with Rayleigh's dissipation function. We then linearized the nonlinear model around the upright balance point to create a fourth-order state-space system. We included motor dynamics using a differential relationship between voltage, current, and back-EMF. We then analyzed the open-loop characteristics of the entire model using block diagram reduction, the Routh–Hurwitz criterion, and frequency response methods. We used metaheuristic optimizers like the Genetic Algorithm (GA) and Clonal Selection Algorithm (CSA). We also used traditional methods such as Ziegler–Nichols and MATLAB's auto-tuner to design and fine-tune P, PI, and PID controllers. We improved the gains with ITAE and ISE cost functions, which included versions that penalized undershoot. The results showed that PID controllers worked better than other types, particularly those tuned with GA and CSA. These sophisticated techniques maintained acceptable settling times while achieving zero overshoot and undershoot below 1%. Interestingly, the best overall performance was demonstrated by the GA-tuned PID with a penalized

*Corresponding Team Leader: 2010155@me.buet.ac.bd

ITAE objective. This demonstrates the advantages of combining contemporary optimization with classical control.
Keywords: Self-Balancing Robot, PID Controller, Genetic Algorithm (GA), Clonal Selection Algorithm (CSA), State-Space Modelling

---

1. Introduction

Two-wheeled self-balancing robots (TWSBRs) represent a remarkable intersection of dynamic control and mobile robotics. Mimicking the physics of an inverted pendulum, these robots are inherently unstable and rely entirely on feedback control systems to maintain their balance and respond to movement commands. Due to their low footprint space and high mobility, they are appropriate for a variety of applications, ranging from autonomous transportation and robotic services to education as well as research. On a functional level, the robot's capability to be in an upright posture, whether when moving or not moving, depends on its sensor's capability to estimate angular velocity as well as orientation, as well as its actuator's capability to react with balancing forces in real time. This challenge of dynamic stabilization makes TWSBRs ideal testbeds for developing and benchmarking control algorithms.

With time, researchers have gone on to propose various mathematical models as well as control methods for tackling nonlinear behavior as well as underactuation of such systems. Mahler et al. (2013) [1] developed a complete model with Lagrangian mechanics and provided full-state feedback with a PI controller for stabilization. Frankovský et al. (2017) [2] similarly utilized a Lagrangian approach with a linearized transfer function utilized for a linear state-feedback controller. Le et al. (2024) [3] took an extra step utilizing a nonlinear model from Euler-Lagrange formulation with implementation of Hierarchical and Sliding Mode Control (HSMC and SMC) with trajectory planning from Rapidly-exploring Random Trees (RRT). On the contrary, Halawa et al. (2023) [4] utilized a simplified transfer function with the implementation of a PID controller on a physical robot with an emphasis on practical prototyping. Parallel to these approaches of modeling, recent literature paid more emphasis on metaheuristic optimizers for tuning classical controllers. Kumar et al. (2012) [5] showed tuning of PID with a Genetic Algorithm (GA) significantly enhanced response characteristics for a neighborhood of equilibrium for an inverted pendulum system. Olmez et al. (2022) [6] utilized an extensive range of optimizers-Clonal Selection Algorithm (CSA), PSO,

ABC, and DE, as well as Cuckoo search-on a two-wheeled self-balancing robot, with CSA performing significantly well. Correspondingly, Hassanzadeh and Mobayen (2011) [7] compared GA, PSO, and ACO in rotary pendulum systems, with Yetayew and Tesfaye (2024) [8] comparing GA and PSO as well as Grey Wolf Optimizer (GWO) in sliding mode control for Segway-style robots. Each of these papers insists repeatedly on intelligent tuning for enhancing transient as well as steady-state control parameters.

Despite all of these advances, though, very few directly treat classical controllers of P, PI, or PID type operating in addition with current optimization techniques. More specifically, although Mahler et al.'s paper [1] was thorough in its coverage of modeling and full-state design of PI control, it did not address nor compare simpler PID-type setups fine-tuned with intelligent algorithms. This creates a valuable gap in understanding: could classical controllers, optimally tuned, offer similar efficiency as more advanced setups?

To respond to this, our research performs a comparative study of P, PI, and PID controllers of a two-wheeled self-balancing robot with an emphasis on tuning using metaheuristic algorithms, namely Genetic Algorithm (GA) and Clonal Selection Algorithm (CSA). Through an analysis of the transient as well as steady-state response of the system for each of the control structures, we intend to get a clear picture of what these conventional controllers can offer when optimally optimized in an intelligent way. This paper not only compares their efficiency but also showcases the effectiveness and applicability of classical control when aided with advanced computing methods.

2. Methodology

The methodology for this study was structured in two primary phases: system analysis and controller design.

The two-wheeled self-balancing robot is modeled as a mobile inverted pendulum system consisting of two primary masses: the upper body mass m_H and the lower motor mass m_L. These are abstracted as point masses separated by a rigid link of length l. The system rotates about the axle connecting the wheels, with the entire dynamics influenced by gravitational, inertial, and frictional forces.

From this model, the open-loop system's block diagram was derived and subsequently simplified using standard block diagram reduction techniques. A comprehensive analysis of the uncom-

pensated system was then performed. This involved evaluating its steady-state error in response to step, ramp, and parabolic inputs to characterize its inherent performance limitations. The stability of the open-loop system was formally assessed using the Routh-Hurwitz stability criterion, and its frequency response characteristics were visualized with a polar plot.

Following the characterization of the plant, the focus shifted to the design and tuning of controllers within a unity feedback configuration. Proportional (P), Proportional-Integral (PI), and Proportional-Integral-Derivative (PID) controllers were systematically optimized using a variety of methods to ensure a robust comparison. The tuning process included a classical approach, the Ziegler-Nicholas method, as well as advanced metaheuristic techniques, namely the Genetic Algorithm (GA) and the Clonal Selection Algorithm (CSA). To provide a baseline for comparison, MATLAB's automated PID tuning software was also utilized. A detailed comparative analysis of the performance achieved by each controller and tuning method is presented in Section 3.

2.1. Mathematical Modelling

Following the approach by Mahler et al. (2013) [1], Lagrangian mechanics is employed to derive the nonlinear equations of motion, where the generalized coordinates are the translational displacement s of the wheels and the angular pitch ϕ of the body.

The Lagrangian of the system is defined as:

$$\mathscr{L} = T - V \tag{1}$$

where T is the total kinetic energy and V is the potential energy of the system.

The kinetic energy is composed of:

$$T_H = \frac{1}{2} m_H \left(\dot{s}^2 + 2l\dot{s}\dot{\phi}\cos\phi + l^2\dot{\phi}^2 \right) \tag{2}$$

$$T_L = \frac{1}{2} m_L \dot{s}^2 \tag{3}$$

$$T_R = \frac{1}{2} J_R \left(\frac{\dot{s}}{r} \right)^2 \tag{4}$$

The potential energy is:

$$V = m_H g l \cos\phi \tag{5}$$

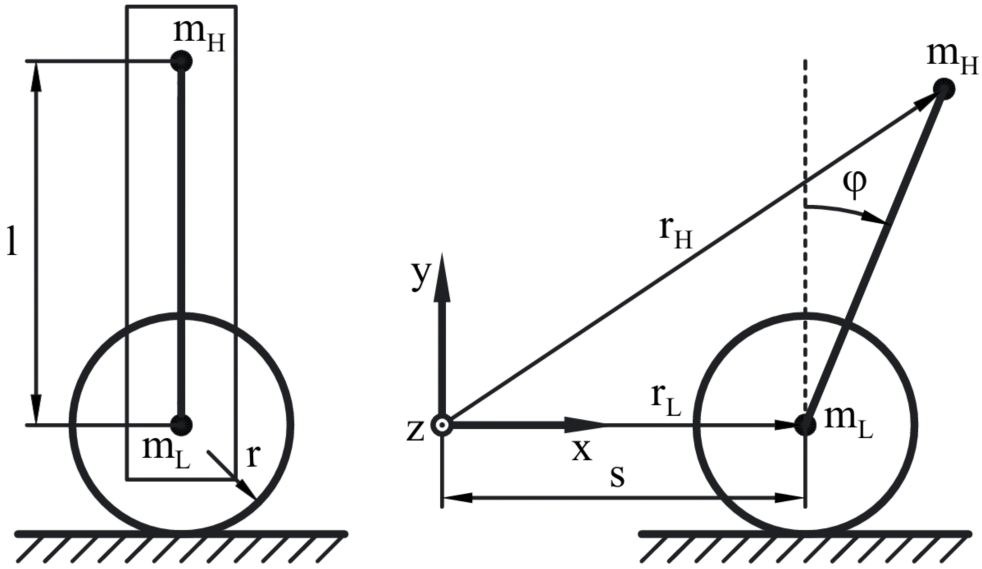


Figure 1: Structure and definitions of the mathematical model

The dissipative effects are modeled using Rayleigh's dissipation function:

$$D = \frac{1}{2}\left(c_v \dot{s}^2 + c_\omega \left(\frac{\dot{s}}{r} - \dot{\phi}\right)^2\right) \tag{6}$$

Applying the Euler–Lagrange equation with dissipation:

$$\frac{d}{dt}\left(\frac{\partial \mathscr{L}}{\partial \dot{q}_i}\right) - \frac{\partial \mathscr{L}}{\partial q_i} + \frac{\partial D}{\partial \dot{q}_i} = \tau_i \tag{7}$$

This yields a set of nonlinear second-order differential equations in terms of ϕ and s, which represent the robot's pitch and wheel motion.

2.2. Motor Dynamics

The motor generates torque according to:

$$M_D = k_M i \tag{8}$$

The current i evolves according to the motor's equivalent circuit:

$$\frac{di}{dt} = \frac{1}{L_M}\left(u_M - R_M i - k_M \left(\frac{\dot{s}}{r} - \dot{\phi}\right)\right) \tag{9}$$

2.3. Linearization

The system is represented by the state vector:

$$x = \begin{bmatrix} \phi & \dot{\phi} & v & i \end{bmatrix}^T, \quad u = u_M \tag{10}$$

The linearized state-space model around the upright equilibrium ($\phi = 0$) is given as:

$$\dot{x} = Ax + Bu \tag{11}$$

The parameters used in the modeling and linearization in the article by Mahler et al. [1] are listed in Table 1.

Table 1: Model parameters of the self-balancing robot

Parameter	**Value**
Body (Pendulum-)length l	0.147 m
Wheel radius r	0.056 m
Upper mass m_H	1.1 kg
Lower mass m_L	0.7 kg
Moment of inertia wheels J_R	$2.2 \cdot 10^{-3}$ kg m^2
Torque coefficient motor k_M	0.167 Nm/A
Armature resistance motor R_M	1.26 Ω
Armature inductance motor L_M	25 mH
Rotational friction coefficient c_ω	$2.6 \cdot 10^{-3}$ Nms/rad
Rolling friction coefficient c_v	2 Ns/m

Where the system matrices are:

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 119.1 & -0.4206 & 17.22 & -27.01 \\ -7.699 & 0.0457 & -2.244 & 2.938 \\ 0 & 6.68 & -119.3 & -50.4 \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ 0 \\ 0 \\ \frac{1}{L_m} \end{bmatrix} \tag{12}$$

Mahler et al. [1] applied a full-state feedback controller using the linearized state-space model. The control law is given by:

$$u = r - \mathbf{k}^T \mathbf{x} = r - (k_\varphi \, \varphi + k_\omega \, \omega + k_v \, v + k_i \, i) \tag{13}$$

The feedback gain vector **k** was calculated using Ackermann's formula. The resulting gains were:

$$k_\varphi = -15.3 \tag{14}$$

$$k_\omega = -1.49 \tag{15}$$

$$k_v = -6.33 \tag{16}$$

$$k_i = 0.4 \tag{17}$$

This state-space model is used to design and evaluate various controllers (P, PI, PID) with gains optimized using metaheuristic algorithms.

2.4. Block Diagrams

From the linearized mathematical model we get,

$$\begin{aligned} & \dot{x} = Ax + Bu \\ \Rightarrow \quad & sX(s) = AX(s) + BU(s) \quad \text{(Taking Laplace Transform)} \\ \Rightarrow \quad & X(s) = \frac{1}{s}\{AX(s) + BU(s)\} \end{aligned} \tag{18}$$

from the full-state feedback controller,

$$\begin{aligned} & u = r - k^T x \\ & \Rightarrow U(s) = R(s) - k^T X(s) \quad \text{[Taking Laplacian]} \end{aligned} \tag{19}$$

From these equations we get this block diagram of the system

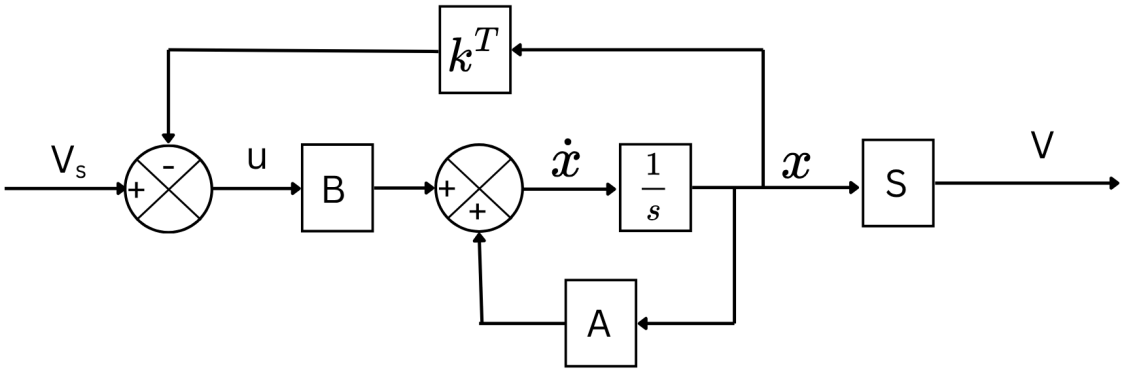


Figure 2: Block diagram of the system

After integrating a controller in the system, we get

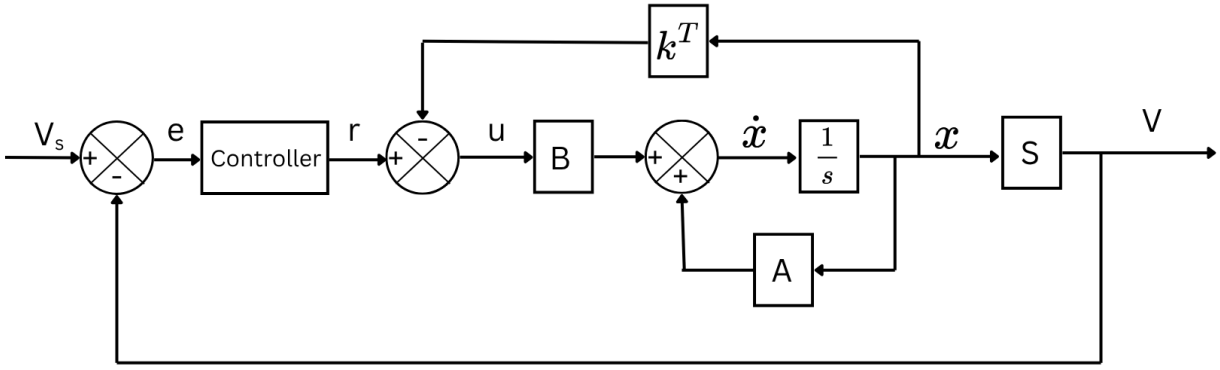


Figure 3: Block diagram of the system with controller

3. System response without controller

3.1. Block diagram reduction

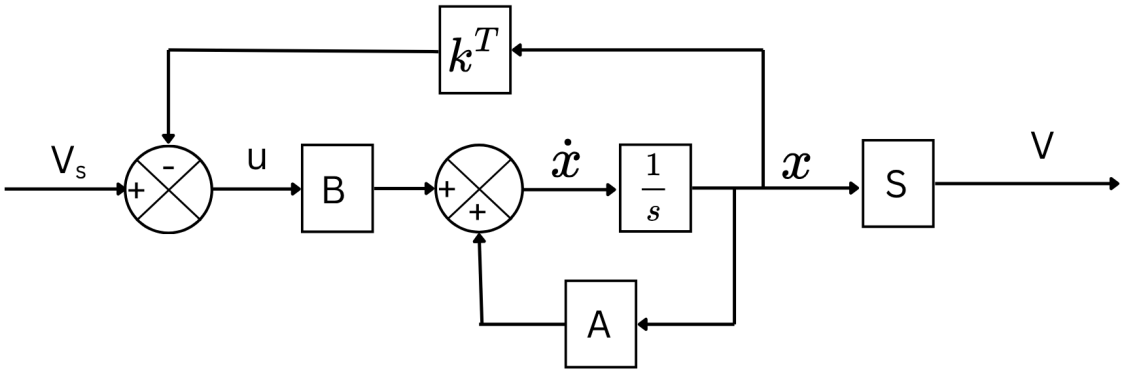
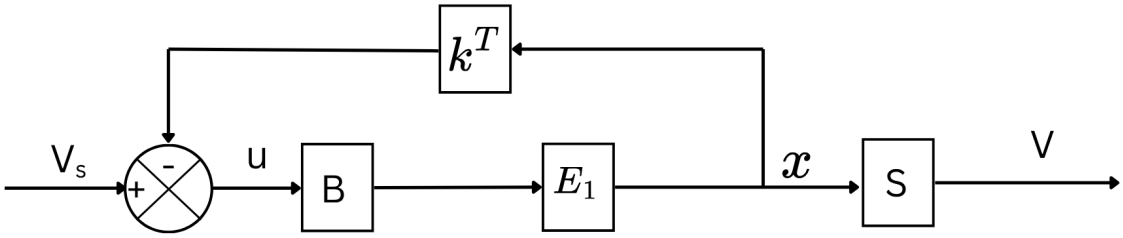


Figure 4: Block diagram without controller

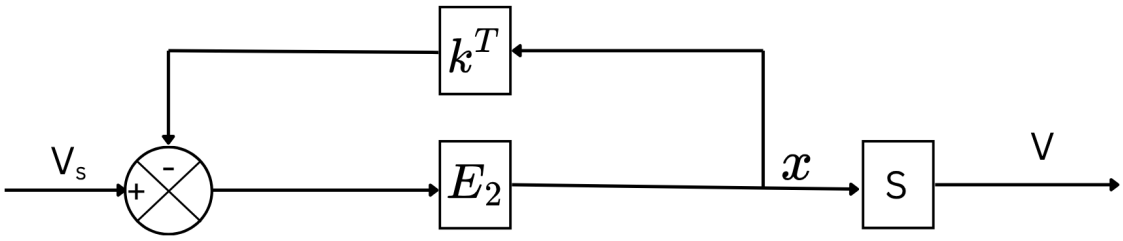
This can be reduced to



where,

$$E_1 = \frac{1}{s}(I - \frac{1}{s}A)^{-1} \tag{20}$$

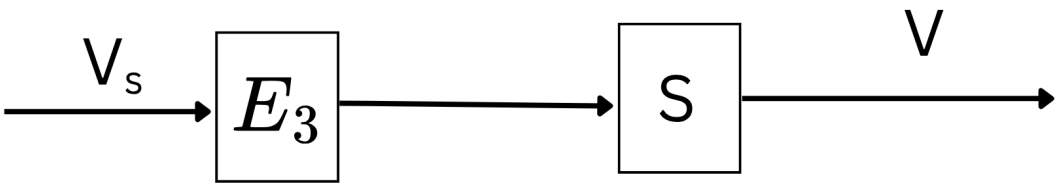
which can be reduced to



where,

$$E_2 = E_1 \times B = \begin{bmatrix} -\dfrac{1080.4(s+0.370903)}{s^4+53.0646s^3+546.283s^2-6062.33s-23724.6} \\ -\dfrac{1080.4s(s+0.370903)}{s^4+53.0646s^3+546.283s^2-6062.33s-23724.6} \\ \dfrac{117.52\left(s^2+0.000464874s-48.3206\right)}{s^4+53.0646s^3+546.283s^2-6062.33s-23724.6} \\ \dfrac{40\left(s^3+2.6646s^2-118.943s-134.684\right)}{s^4+53.0646s^3+546.283s^2-6062.33s-23724.6} \end{bmatrix} \tag{21}$$

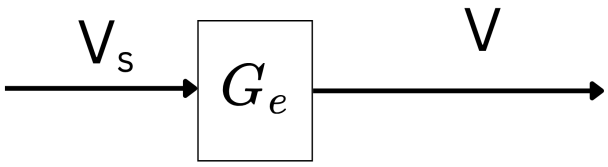
This can be reduced to



where

$$E_3 = \frac{E_2}{(1+k^T * E_2)} = \begin{bmatrix} -\dfrac{1080.4(s+0.370903)}{s^4+69.0646s^3+1454.811s^2+9161.434s+16197.299} \\ -\dfrac{1080.4s(s+0.370903)}{s^4+69.0646s^3+1454.811s^2+9161.434s+16197.299} \\ \dfrac{117.52\left(s^2+0.000464874s-48.3206\right)}{s^4+69.0646s^3+1454.811s^2+9161.434s+16197.299} \\ \dfrac{40\left(s^3+2.6646s^2-118.943s-134.684\right)}{s^4+69.0646s^3+1454.811s^2+9161.434s+16197.299} \end{bmatrix} \tag{22}$$

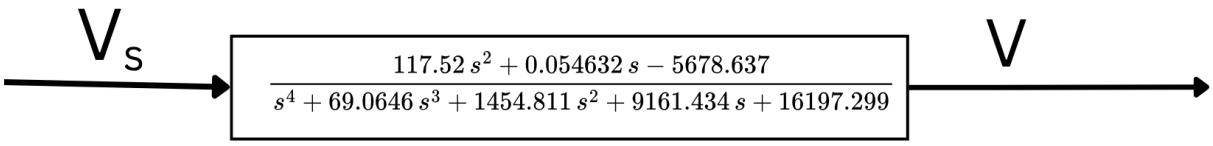
This can be reduced to



Where

$$G_e = S \times E_3 = \frac{117.52\,s^2 + 0.054632\,s - 5678.637}{s^4 + 69.0646\,s^3 + 1454.811\,s^2 + 9161.434\,s + 16197.299} \tag{23}$$

So the final reduced form is



3.2. Block diagram with controller

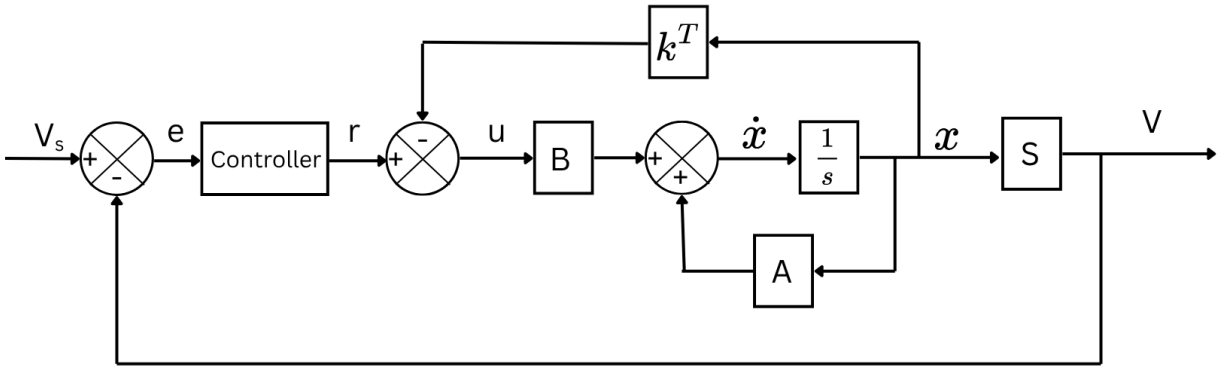


Figure 5: Block diagram with controller

Which is equivalent to

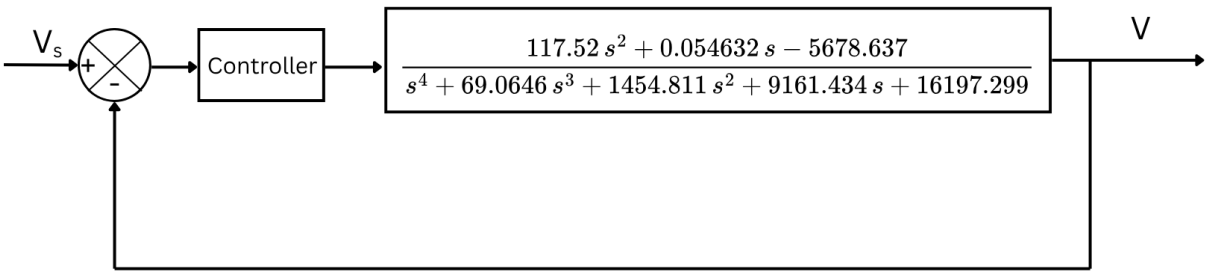


Figure 6: reduced block diagram with controller

3.3. Stability Analysis (without Controller)

Block diagram of the system (without controller) is:

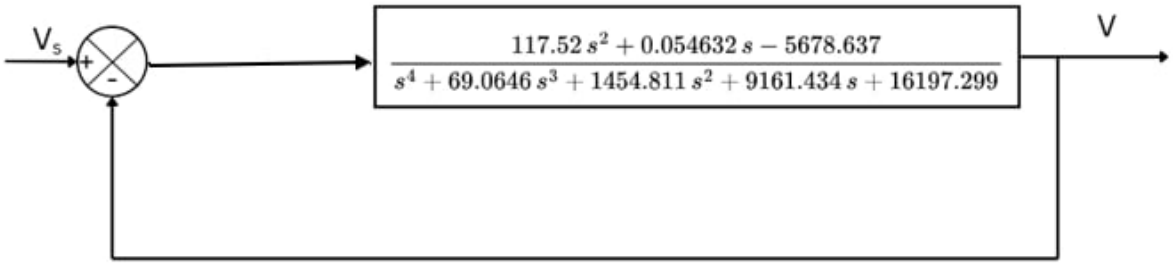


Figure 7: Block diagram for stability analysis

Hear,

$$G(s) = \frac{117.52\,s^2 + 0.054632\,s - 5678.637}{s^4 + 69.0646\,s^3 + 1454.811\,s^2 + 9161.434\,s + 16197.299} \tag{24}$$

The closed loop Transfer function will be :

$$G_e(s) = \frac{117.52\,s^2 + 0.054632\,s - 5678.637}{s^4 + 69.0646\,s^3 + 1572.331\,s^2 + 1961.488\,s + 10518.662} \tag{25}$$

The characteristics equation will be :

$$s^4 + 69.0646\,s^3 + 1572.331\,s^2 + 1961.488\,s + 10518.662 = 0 \tag{26}$$

Routh Table :

Power of s	Column 1	Column 2	Column 3
s^4	1	1572.331	10518.662
s^3	69.0646	1961.488	0
s^2	1543.93	10518.662	0
s^1	1490.95	0	0
s^0	10518.662	0	0

There is no sign change. So the system is stable.

3.4. Steady State error Analysis (without controller)

- **For Unit Step Input:**

$$K_p = \lim_{s\to 0} G(s) = \lim_{s\to 0} \frac{117.52s^2 + 0.054632s - 5678.637}{s^4 + 69.06466s^3 + 1454.811s^2 + 9161.434s + 16197.299} = \frac{-5678.637}{16197.299} = -0.35$$

$$e_{s.s}(\text{unit step input}) = \frac{1}{1+K_p} = \frac{1}{1-0.35} = \frac{20}{13}$$

- **For Ramp Input:**

$$K_v = \lim_{s\to 0} sG(s) = 0$$

$$e_{s.s}(\text{ramp input}) = \frac{1}{K_v} = \frac{1}{0} = \infty$$

- **For Parabolic Input:**

$$K_a = \lim_{s\to 0} s^2 G(s) = 0$$

$$e_{s.s}(\text{parabolic input}) = \frac{1}{K_a} = \frac{1}{0} = \infty$$

3.5. Responses (without Controller)

For Unit Step Input :

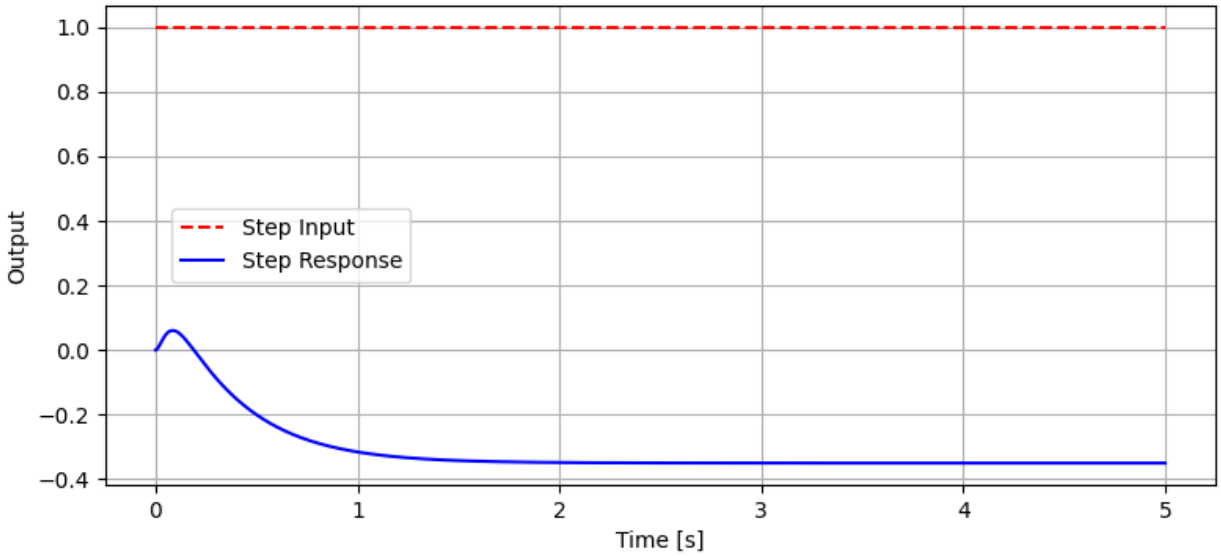


Figure 8: Step input and output

For Ramp Input :

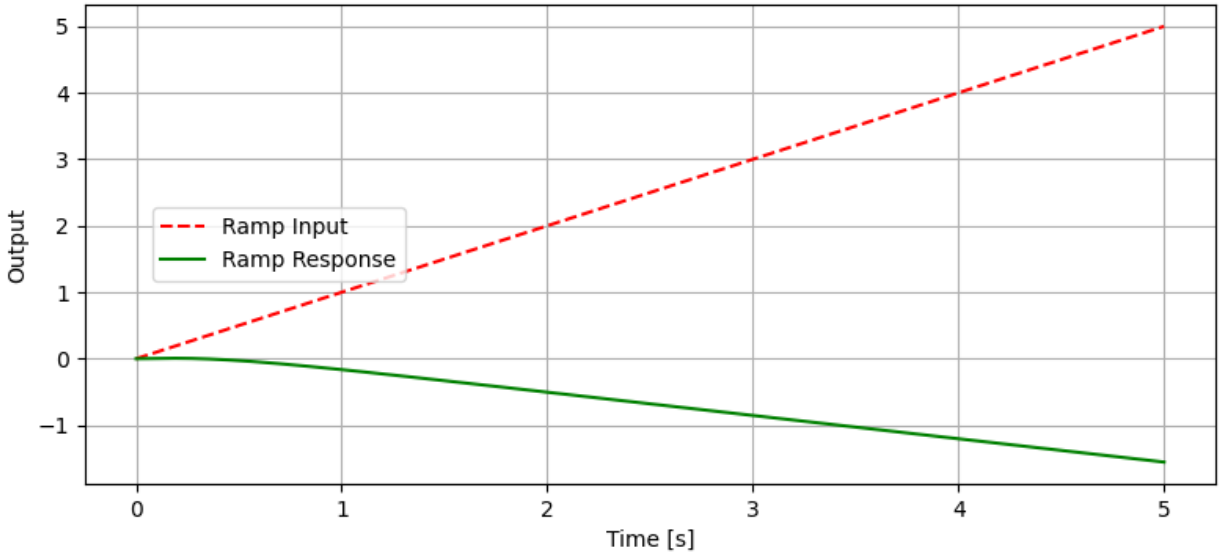


Figure 9: Ramp input and output

For Parabolic Input :

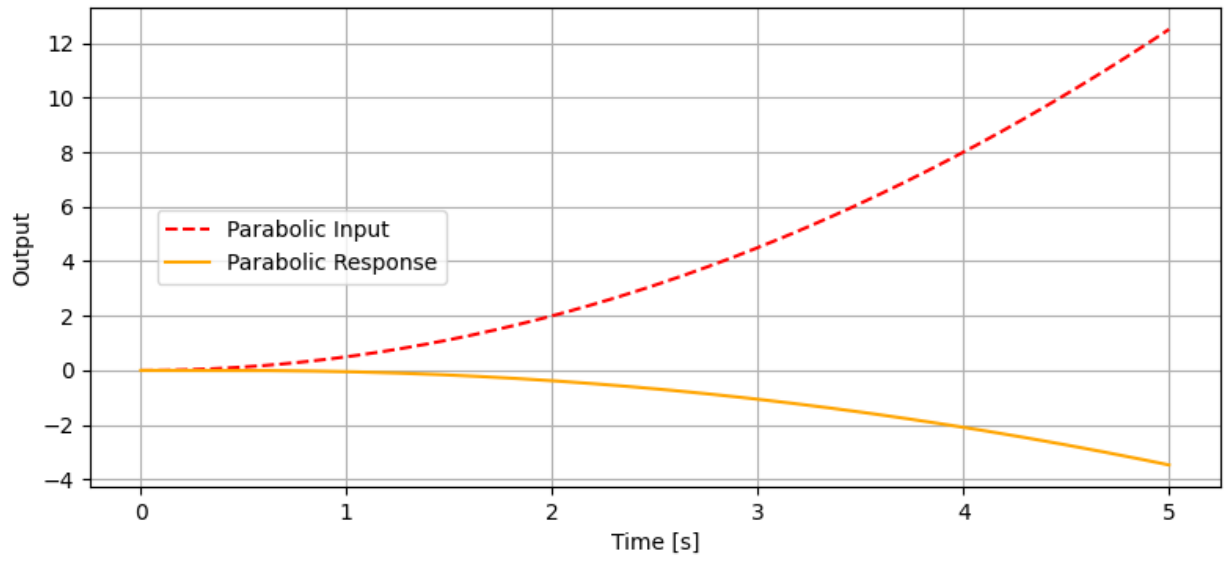


Figure 10: Parabolic input and output

3.6. Bode Plot and Polar Plot (without Controller)

Bode Plot (Magnitude) :

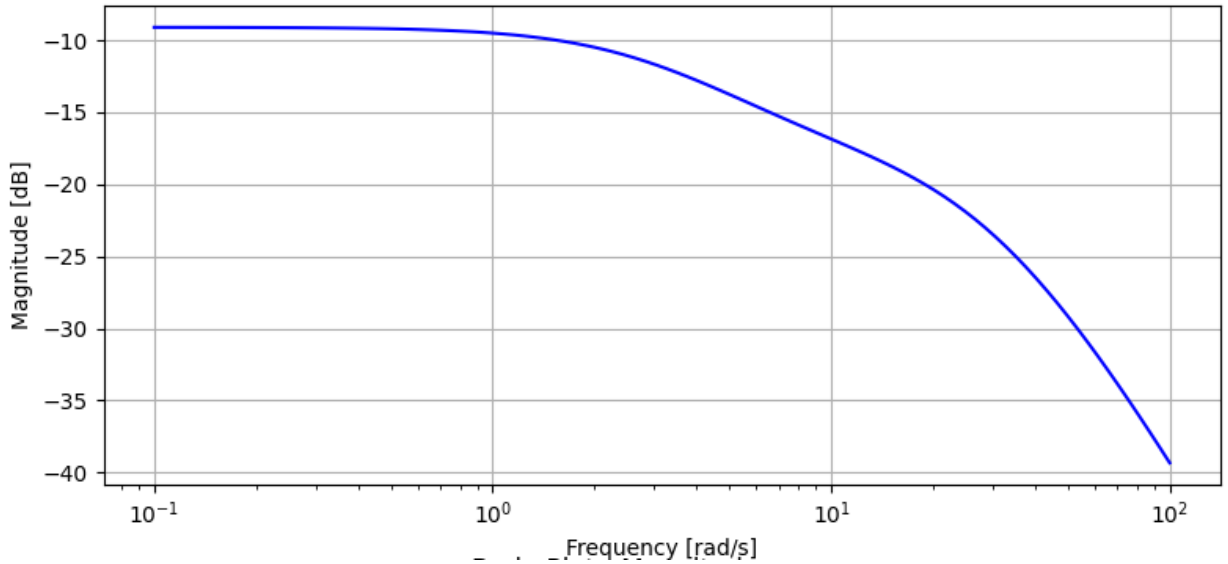


Figure 11: Bode plot of magnitude

Bode Plot (Phase) :

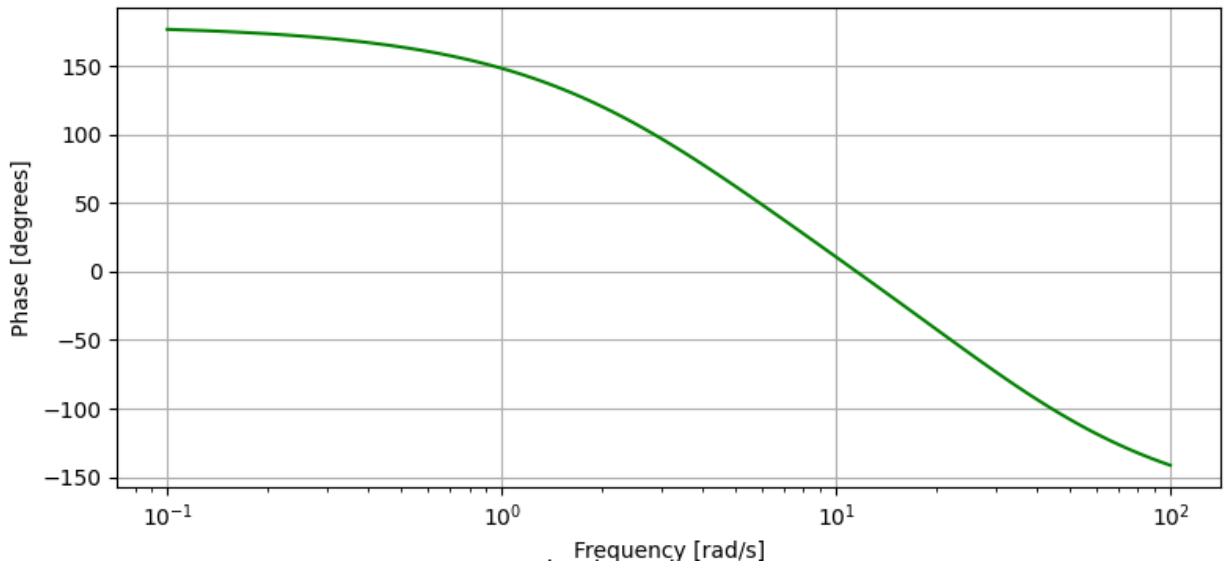


Figure 12: Bode plot of phase

Polar Plot :

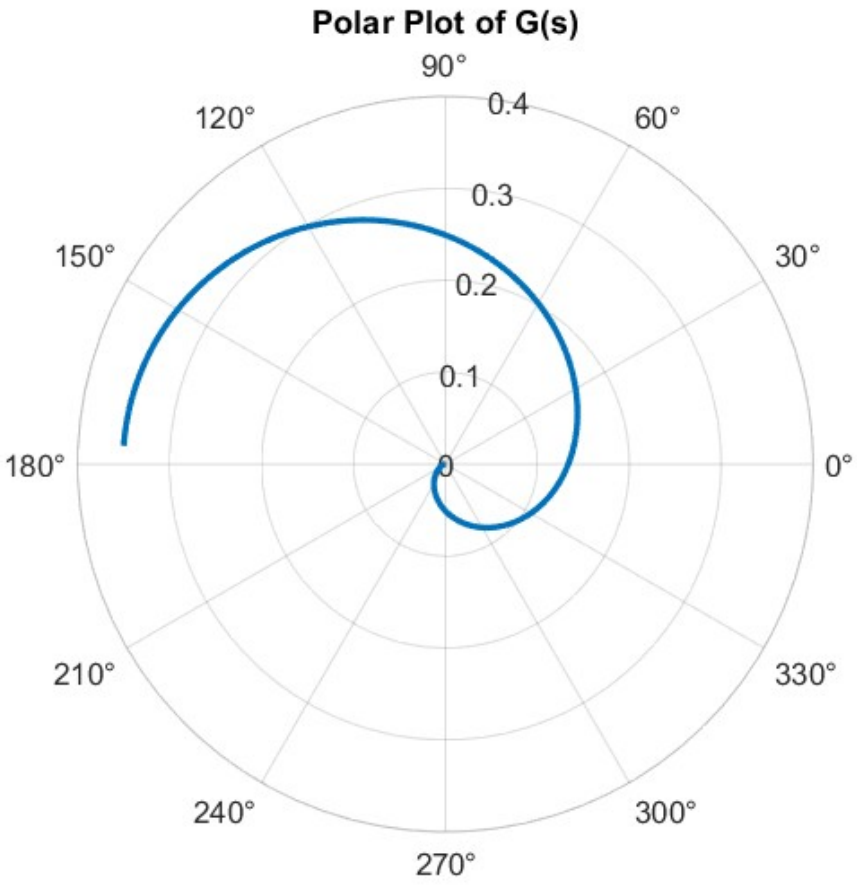


Figure 13: polar plot

3.7. Root locus plot

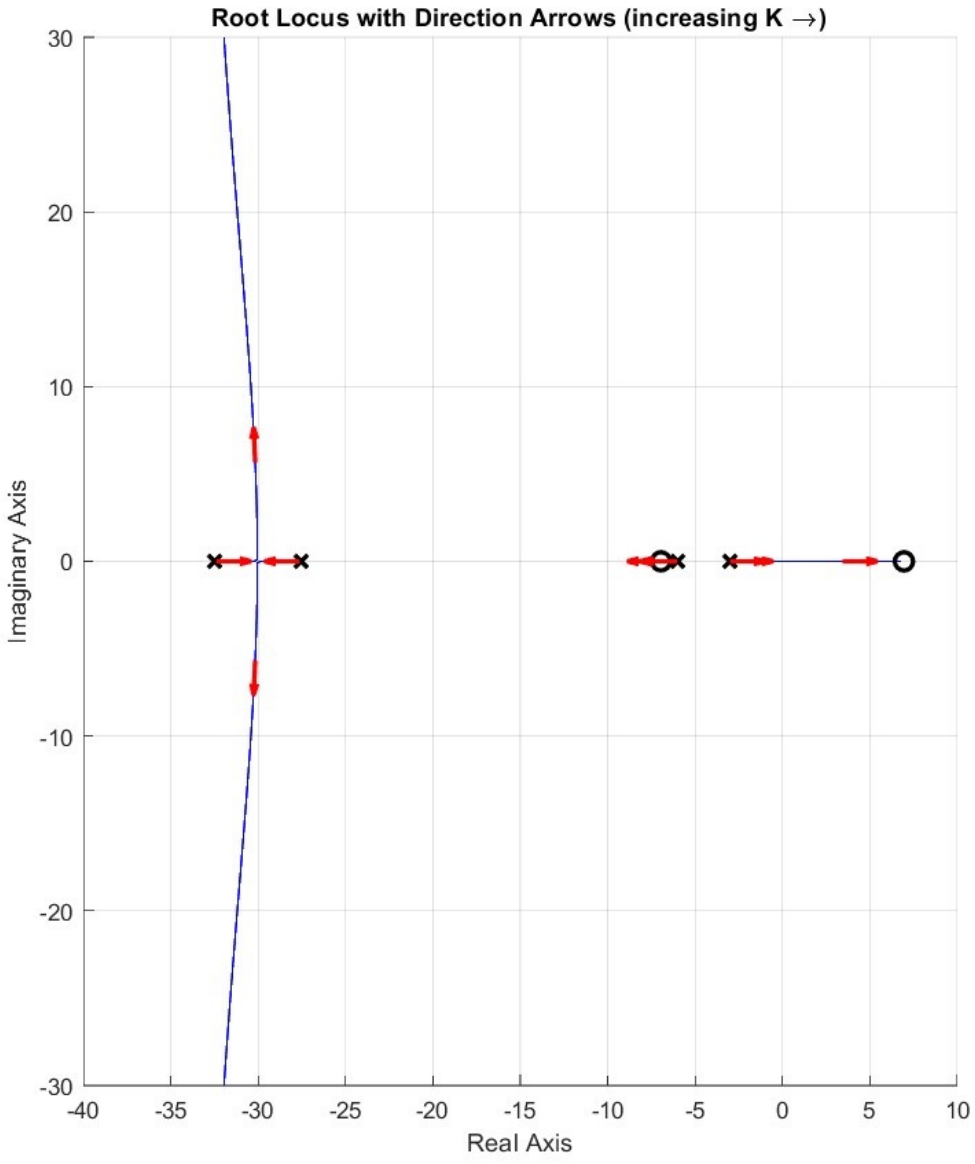


Figure 14: root locus plot

4. Controller Tuning and Comparative Analysis

Following the initial system analysis, this section details the design and optimization of Proportional (P), Proportional-Integral (PI), and Proportional-Integral-Derivative (PID) controllers. To ensure a comprehensive evaluation, several tuning methodologies were employed, including meta-heuristic optimization techniques and industry-standard software tools. The performance of each

tuned controller is evaluated based on its transient response characteristics, such as rise time, settling time, overshoot and undershoot.

4.1. Ziegler-Nichols (ZN) Tuning Method

The Ziegler–Nichols algorithm is a heuristic method for tuning PID controllers based on the system's response to controller gain changes. The algorithm involves setting the integral and derivative gains to zero, then gradually increasing the proportional gain until the system output exhibits sustained oscillations with marginal stability. The gain at this point is called the critical gain (K_{cr}), and the oscillation frequency and period is the critical frequency (ω_{cr}) and critical period (P_{cr}) respectively. These values are then used in empirical formulas for calculating initial PID parameters, which are subsequently refined through further tuning.

The PID controller transfer function can be written as

$$K_P + \frac{K_I}{s} + K_D s$$

With $T_i = \frac{K_P}{K_I}$ and $T_d = \frac{K_D}{K_P}$ it can be rewritten as,

$$K_P(1 + \frac{1}{T_i s} + T_d s)$$

With $K_I = 0$ and $K_D = 0$ the characteristic equation of the closed-loop transfer function with the controller present can be simplified to,

$$\begin{aligned} &s^4 + 69.0646s^3 + (117.5K_P + 1454.8)s^2 \\ &+(0.05463K_P + 9161.434)s - (5678.637K_P - 16197.3) = 0 \end{aligned} \tag{27}$$

Substituting $s = j\omega$ to solve for marginal stability,

$$\begin{aligned} &\omega^4 - 69.0646j\omega^3 - (117.52K_P + 1454.811)\omega^2 \\ &+(0.054632K_p j + 9161.434j)\omega - (5678.637K_P - 16197.299) = 0 \end{aligned} \tag{28}$$

Equating real and imaginary coefficients as 0 and solving subsequent system of equations for

real values of K_P and positive ω,

$$\omega^4 - (117.52K_P + 1454.811)\omega^2 - (5678.637K_P - 16197.299) = 0 \tag{29}$$

$$-69.0646\omega^3 + (0.054632K_P + 9161.434)\omega = 0 \tag{30}$$

$$K_P = K_{cr} = -7.48486$$

$$\omega = \omega_{cr} = 11.51713\ rad/sec$$

From the value of ω, the critical oscillation period can thus be calculated as,

$$P_{cr} = \frac{2\pi}{\omega_{cr}} = 0.54555\ sec$$

The values of K_{cr} and P_{cr} can now be used in conjunction with empirical relations obtained from the Ziegler-Nichols tuning table as illustrated in Table 2 for yielding the controller parameters for proportional (P), proportional-integral (PI), and proportional-integral-derivative (PID) configurations.

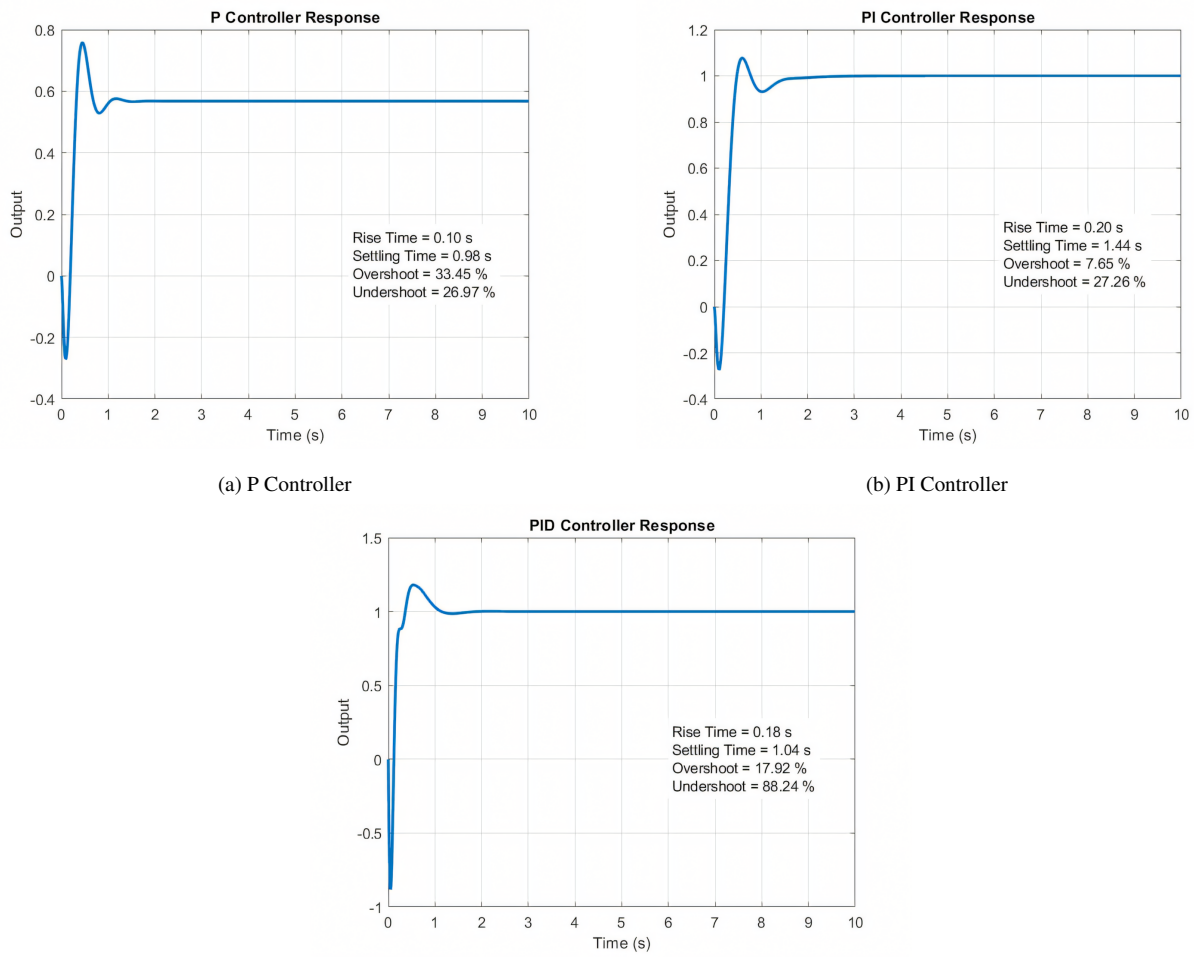
Table 2: Ziegler-Nichols Tuning Rule Based on Critical Gain K_{cr} and Critical Period P_{cr}

Type of Controller	K_P	T_i	T_d
P	$0.5K_{cr}$	∞	0
PI	$0.45K_{cr}$	$\frac{1}{1.2}P_{cr}$	0
PID	$0.6K_{cr}$	$0.5P_{cr}$	$0.125P_{cr}$

As shown in Table 3, the controller gains were calculated along with their performance measures. In addition, Figure 15 illustrates the dynamic unit step response of each controller tuned using the Ziegler-Nichols approach.

Table 3: ZN Parameter and Performance Comparison for P, PI and PID controller Tuning

Parameter	**P**	**PI**	**PID**
K_P	-3.7424	-3.3682	-4.4909
K_I	0	-7.4087	-16.4638
K_D	0	0	-0.3063
Performance Metrics			
Rise Time (s)	*0.10*	*0.20*	*0.18*
Settling Time (s)	*0.98*	*1.44*	*1.04*
Overshoot (%)	*33.45*	*7.65*	*17.92*
Undershoot (%)	*26.97*	*27.26*	*88.24*



(a) P Controller

(b) PI Controller

(c) PID Controller

Figure 15: Comparison of step responses for ZN-tuned P, PI and PID controller.

4.2. Genetic Algorithm (GA) Based Tuning

The Genetic Algorithm (GA) is a stochastic search heuristic inspired by the process of natural selection. In this work, GA was employed to find the optimal set of controller gains (K_P, K_I, K_D) by minimizing a predefined cost function over a series of generations. The primary cost function utilized was the Integral of Time-weighted Absolute Error (ITAE), given by the formula:

$$\text{ITAE} = \int_0^\infty t|e(t)|dt \tag{31}$$

This metric is particularly effective at penalizing errors that persist over time, leading to responses with low settling times and minimal oscillations. For specific analyses requiring zero undershoot, the cost function was augmented with a high-value penalty term that was activated if the system

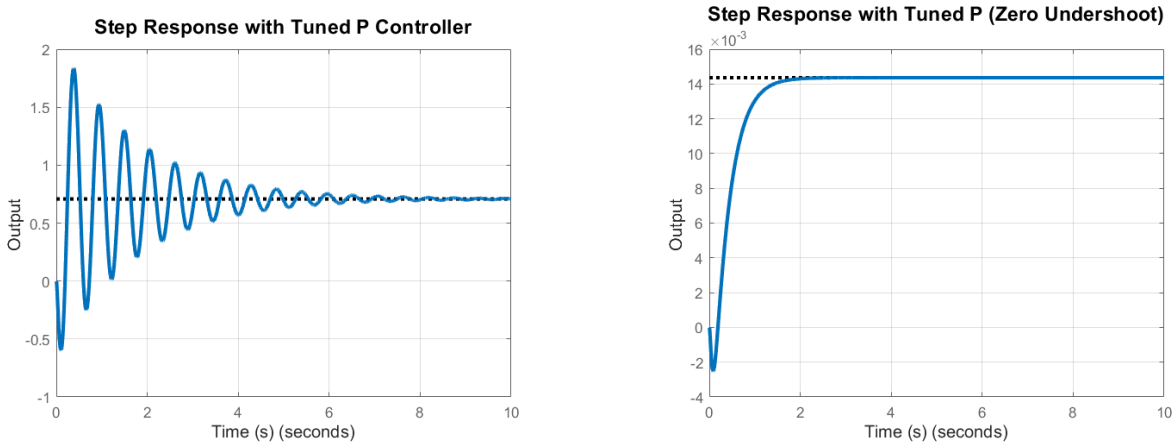
response dropped below the setpoint, thereby guiding the algorithm towards solutions that strictly avoided undershoot.

4.2.1. P Controller Tuning with GA

The GA was first configured to optimize a P-only controller. A comparison was performed between tuning with the standard ITAE cost function and tuning with the ITAE cost function augmented with a penalty to eliminate undershoot. The parameters for the optimization runs and the resulting optimal gain are detailed in Table 4. The resulting step responses are shown side-by-side in Figure 16.

Table 4: GA Parameter and Performance Comparison for P Controller Tuning

Parameter	**Standard ITAE**	**ITAE + Undershoot Penalty**
K_P Range	[-10, 10]	[-10, 10]
Generations	100	100
Population Size	80	80
Crossover Rate	0.8	0.8
Mutation Rate	0.1	0.1
Optimal K_P	*-6.9230*	*-0.0416*
Performance Metrics		
Rise Time (s)	*0.0425*	*0.7436*
Settling Time (s)	*7.9264*	*1.4967*
Overshoot (%)	*159.2372*	*0.0000*
Undershoot (%)	*59.5557*	*0.2522*



(a) Standard ITAE cost function.

(b) ITAE with undershoot penalty.

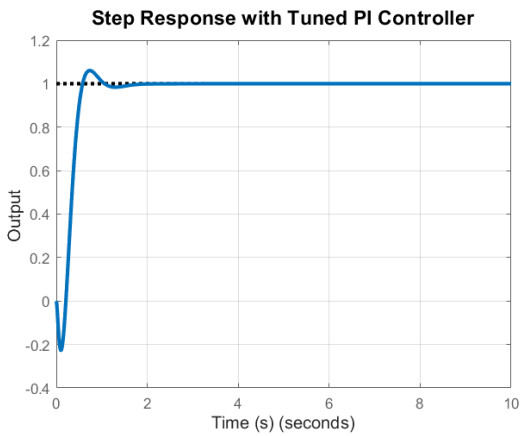
Figure 16: Comparison of step responses for GA-tuned P controller.

4.2.2. PI Controller Tuning with GA

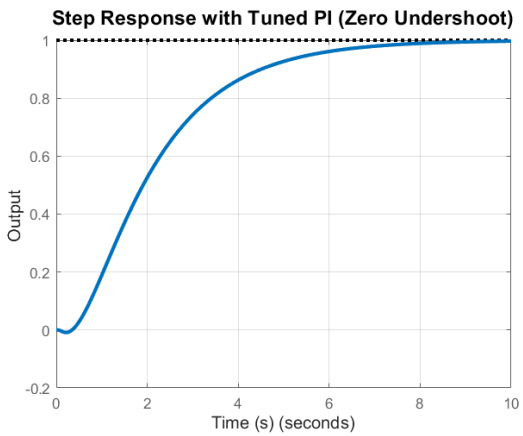
Next, the algorithm was adapted to tune a PI controller, again comparing the results from the standard ITAE cost function against the penalized version. The search space was expanded to include the integral gain K_i. The optimization parameters and resulting performance metrics are listed in Table 5, and the system responses are depicted side-by-side in Figure 17.

Table 5: GA Parameter and Performance Comparison for PI Controller Tuning

Parameter	**Standard ITAE**	**ITAE + Undershoot Penalty**
K_P Range	[-5, 5]	[-5, 5]
K_I Range	[-10, 5]	[-2, 2]
Generations	200	200
Population Size	80	80
Crossover Rate	0.8	0.8
Mutation Rate	0.1	0.15
Optimal K_P	*-2.8186*	*0.0285*
Optimal K_I	*-7.5230*	*-1.2159*
Performance Metrics		
Rise Time (s)	*0.2633*	*3.7564*
Settling Time (s)	*0.9566*	*7.0422*
Overshoot (%)	*6.0572*	*0.0000*
Undershoot (%)	*22.7447*	*0.8409*



(a) Standard ITAE cost function.



(b) ITAE with undershoot penalty.

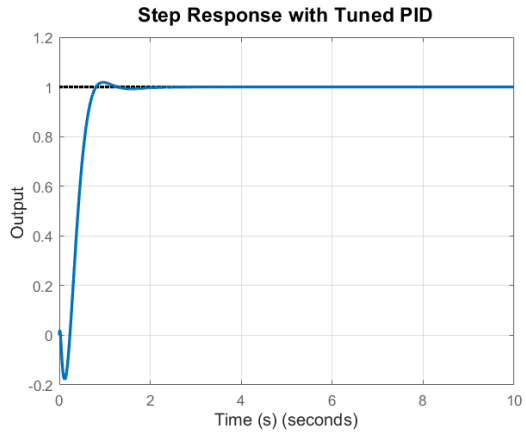
Figure 17: Comparison of step responses for GA-tuned PI controller.

4.2.3. PID Controller Tuning with GA

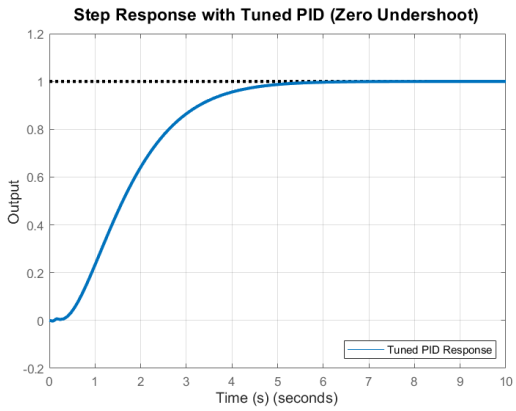
Finally, a full PID controller was optimized using the same comparative approach. The derivative gain K_d was included in the search, and the number of generations was increased to ensure convergence. The parameters and performance results are shown in Table 6, with the corresponding responses in Figure 18.

Table 6: GA Parameter and Performance Comparison for PID Controller Tuning

Parameter	**Standard ITAE**	**ITAE + Undershoot Penalty**
K_P Range	[-10, 10]	[-10, 10]
K_I Range	[-20, 10]	[-5, 5]
K_D Range	[-5, 5]	[-1, 1]
Generations	200	200
Population Size	80	80
Crossover Rate	0.8	0.8
Mutation Rate	0.1	0.1
Optimal K_P	*-2.1769*	*0.0651*
Optimal K_I	*-6.2132*	*-1.5575*
Optimal K_D	*0.0385*	*-0.0200*
Performance Metrics		
Rise Time (s)	*0.3722*	*2.5945*
Settling Time (s)	*0.7537*	*4.6508*
Overshoot (%)	*1.8503*	*0.0000*
Undershoot (%)	*17.9021*	*0.2404*



(a) Standard ITAE cost function.



(b) ITAE with undershoot penalty.

Figure 18: Comparison of step responses for GA-tuned PID controller.

4.3. Clonal Selection Algorithm (CSA) Based Tuning

The Clonal Selection Algorithm (CSA) is another bio-inspired optimization technique, mimicking the affinity maturation process of the immune system's B-cells. An initial investigation was conducted to determine the most effective cost function for this algorithm. When allowing for undershoot, the Integral of Squared Error (ISE) cost function was found to produce a superior transient response compared to ITAE. However, when the objective was modified to completely eliminate undershoot by adding a penalty term, the ITAE-based cost function yielded better results. Therefore, the following analysis presents a comparison between the best achievable response with undershoot (using standard ISE) and the best achievable response with zero undershoot (using penalized ITAE). The Integral Squared Error (ISE) cost function, defined as:

$$\text{ISE} = \int_0^\infty e(t)^2 dt \tag{32}$$

The Integral of Time-weighted Absolute Error (ITAE), difined as:

$$\text{ITAE} = \int_0^\infty t|e(t)|dt \tag{33}$$

The core parameters for the CSA, shown in Table 7, were held same across all tuning scenarios. The only variables were the controller structure (P, PI, or PID) and the specific cost function being optimized. The following analysis presents a comparison between the best achievable response with undershoot (using standard ISE) and the best achievable response with zero undershoot (using penalized ITAE).

Table 7: Core Parameters for Clonal Selection Algorithm

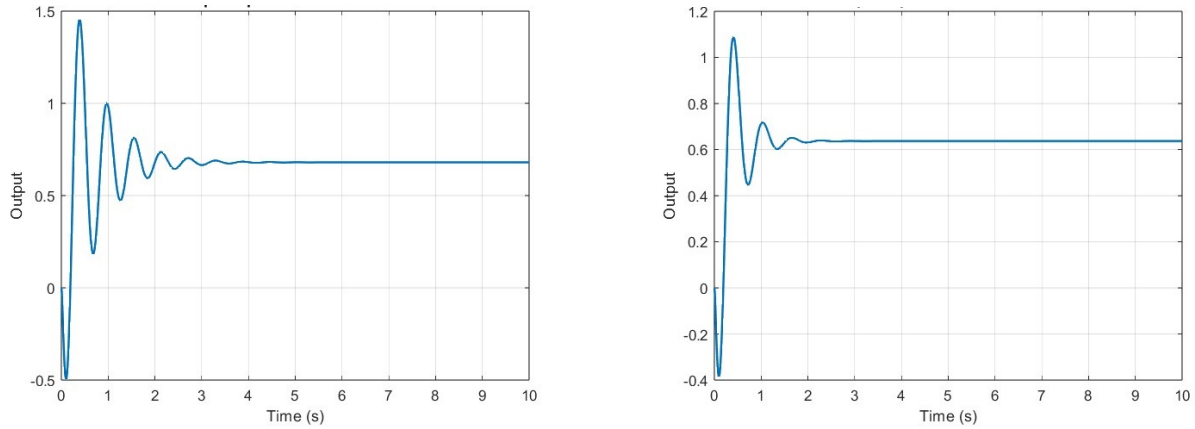
Parameter	**Value**
Population Size	30
Cloning Factor	5
No. of Selected Clones	5
No. of Random Inserts	2
Base Mutation Rate	0.2
Maximum Generations	100

4.3.1. P Controller Tuning with CSA

The CSA was first applied to tune a P-only controller. The optimization results are detailed in Table 8, and the corresponding step responses are shown side-by-side in Figure 19.

Table 8: CSA Performance Comparison for P Controller Tuning

Parameter	**Standard ISE (with Undershoot)**	**Penalized ITAE (Zero Undershoot)**
K_P Range	*[-10, 10]*	*[-10, -5]*
Optimal K_P	*-6.055873*	*-5.000000*
Performance Metrics		
Rise Time (s)	*0.0521*	*0.0686*
Settling Time (s)	*3.0414*	*1.7083*
Overshoot (%)	*113.90*	*70.79*
Undershoot (%)	*49.53*	*38.53*



(a) Response with standard ISE.

(b) Response with penalized ITAE.

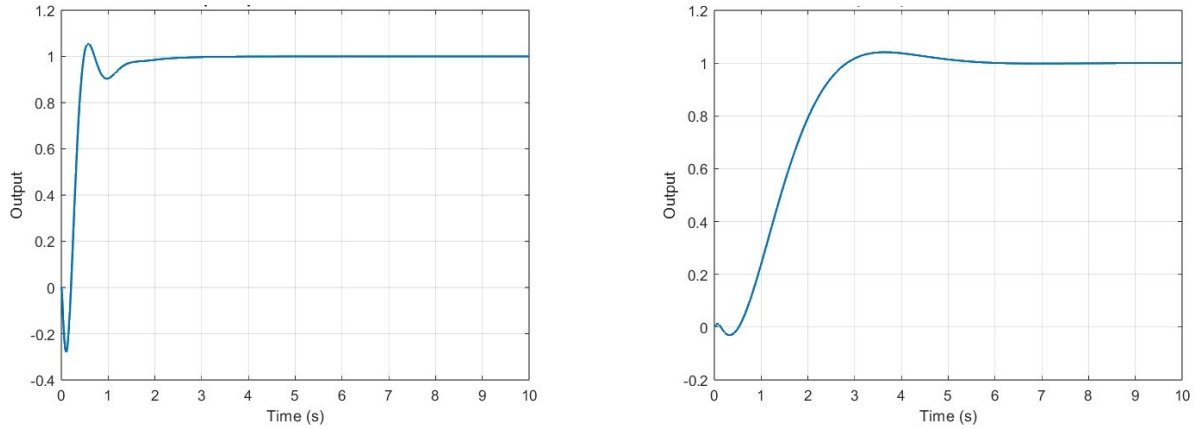
Figure 19: Comparison of step responses for CSA-tuned P controller.

4.3.2. PI Controller Tuning with CSA

The CSA was subsequently used to tune a PI controller. The results are presented in Table 9 and Figure 20.

Table 9: CSA Performance Comparison for PI Controller Tuning

Parameter	**Standard ISE (with Undershoot)**	**Penalized ITAE (Zero Undershoot)**
K_P Range	*[-10, 10]*	*[0.3, 5]*
K_I Range	*[-10, 10]*	*[-3, 5]*
Optimal K_P	*-3.473711*	*0.300000*
Optimal K_I	*-6.903409*	*-2.005857*
Performance Metrics		
Rise Time (s)	*0.1996*	*1.5601*
Settling Time (s)	*1.7592*	*4.7255*
Overshoot (%)	*5.47*	*4.11*
Undershoot (%)	*27.92*	*3.03*



(a) Response with standard ISE.

(b) Response with penalized ITAE.

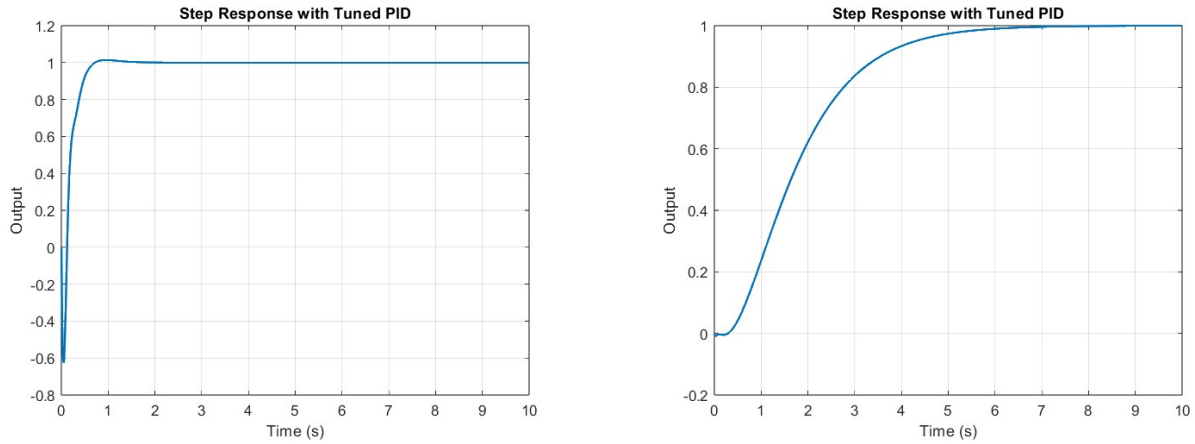
Figure 20: Comparison of step responses for CSA-tuned PI controller.

4.3.3. PID Controller Tuning with CSA

Finally, the CSA was used to optimize a full PID controller. The comparative results are shown in Table 10 and Figure 21.

Table 10: CSA Performance Comparison for PID Controller Tuning

Parameter	**Standard ISE (with Undershoot)**	**Penalized ITAE (Zero Undershoot)**
K_P Range	*[-10, 10]*	*[-5, 5]*
K_I Range	*[-10, 10]*	*[-5, 5]*
K_D Range	*[-10, 10]*	*[-5, 5]*
Optimal K_P	*-3.773710*	*0.011320*
Optimal K_I	*-9.662423*	*-1.481346*
Optimal K_D	*-0.246369*	*-0.006500*
Performance Metrics		
Rise Time (s)	*0.3383*	*2.8752*
Settling Time (s)	*0.6286*	*5.2749*
Overshoot (%)	*1.37*	*0.00*
Undershoot (%)	*62.52*	*0.78*



(a) Response with standard ISE.

(b) Response with penalized ITAE.

Figure 21: Comparison of step responses for CSA-tuned PID controller.

4.4. MATLAB Automated PID Tuner

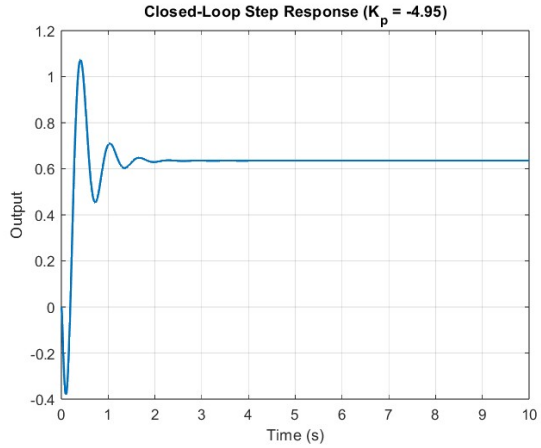
To establish a performance benchmark against a widely used industry tool, the controller was also tuned using the automated tuning application within MATLAB's Control System Toolbox. This tool was used to generate optimal parameters for P, PI, and PID configurations. It employs a frequency-domain-based method to automatically determine the controller gains that balance performance and robustness.

An interesting outcome of this process was that when tuning for a full PID controller, the auto-tuner returned a derivative gain (K_D) of zero, effectively making its suggested PID controller identical to its suggested PI controller. This indicates that for this specific plant, the tuner did not find a significant benefit in adding a derivative term.

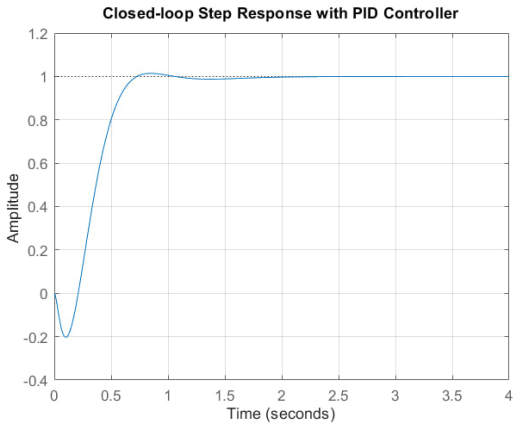
While this method provides a fast and effective way to achieve good rise and settling times, it offers limited direct control over specific response characteristics. For instance, explicitly constraining the system to have zero undershoot is not a standard feature of the auto-tuner, which often prioritizes speed and overall stability. The resulting gains and performance metrics for all three configurations are summarized in Table 11, and their corresponding step responses are plotted side-by-side for comparison in Figure 22.

Table 11: Results from MATLAB Automated Tuner for P, PI, and PID Controllers

Parameter / Metric	**P-Tuned**	**PI-Tuned**	**PID-Tuned**
K_P	-4.95	-2.5568	-2.5568
K_I	0	-6.7867	-6.7867
K_D	0	0	0
Rise Time (s)	0.0695	0.329	0.329
Settling Time (s)	1.6864	0.674	0.674
Overshoot (%)	69.18	1.44	1.44
Undershoot (%)	38.07	20	20



(a) Response of P-tuned controller.



(b) Response of PI/PID-tuned controller.

Figure 22: Comparison of step responses for controllers tuned with MATLAB's auto-tuner.

5. Results and Discussion

This section consolidates the findings from all the tuning methodologies investigated in this paper. A final comparative analysis is conducted to evaluate the effectiveness of the Ziegler-Nicholas method, the Genetic Algorithm (GA), the Clonal Selection Algorithm (CSA), and the MATLAB auto-tuner. The comparison is presented in two parts: first, an evaluation of the controllers tuned with standard cost functions that allow for undershoot, and second, a focused analysis of the controllers specifically optimized to achieve zero undershoot.

5.1. Comparison of Standard Tuning Methods

Table 12 presents a comprehensive summary of the optimal PID controller gains and corresponding transient response metrics for each of the four tuning methods using their standard configurations. For the metaheuristic algorithms, this corresponds to the unpenalized cost functions (ITAE for GA and ISE for CSA). This allows for a direct quantitative comparison of their baseline performance.

Table 12: Performance Comparison of PID Controllers (Without Penalizing Undershoot)

Parameter / Metric	**Ziegler-Nicholas**	**Genetic Algorithm**	**Clonal Selection Alg.**	**MATLAB Auto-Tuner**
Controller Gains				
K_p	*-4.490922*	*-2.1769*	*-3.773710*	*-2.5568*
K_i	*-16.46375*	*-6.2132*	*-9.662423*	*-6.7867*
K_d	*-0.3062543278*	*0.0385*	*-0.246369*	*0*
Performance Metrics				
Rise Time (s)	*0.1773*	*0.3722*	*0.3383*	*0.329*
Settling Time (s)	*1.0384*	*0.7537*	*0.6286*	*0.674*
Overshoot (%)	*17.92*	*1.8503*	*1.37*	*1.44*
Undershoot (%)	*88.24*	*17.9021*	*62.52*	*20*

To provide a qualitative assessment, the step responses of these four controllers are plotted together on the same axes in Figure 23.

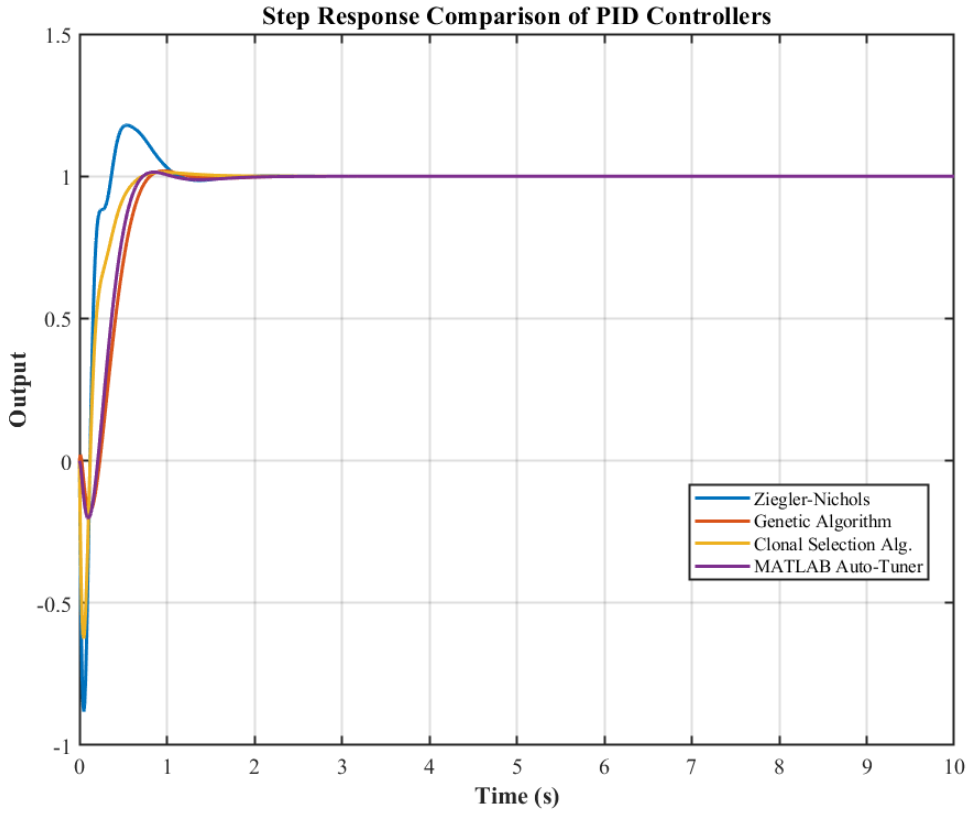


Figure 23: Comparative step response of PID controllers from standard tuning methods.

5.2. Comparison of Zero-Undershoot Tuning Methods

Next, a focused comparison was made between the metaheuristic algorithms when the design objective was explicitly modified to eliminate undershoot. Table 13 shows the results for the GA and CSA when using a penalized cost function. The Ziegler-Nicholas and MATLAB auto-tuner methods are omitted from this direct comparison as they do not offer a native mechanism for penalizing undershoot.

Table 13: Performance Comparison of PID Controllers (Zero-Undershoot Tuning)

Parameter / Metric	**Genetic Algorithm**	**Clonal Selection Alg.**
Controller Gains		
K_p	*0.0651*	*0.011320*
K_i	*-1.5575*	*-1.481346*
K_d	*-0.0200*	*-0.006500*
Performance Metrics		
Rise Time (s)	*2.5945*	*2.8752*
Settling Time (s)	*4.6508*	*5.2749*
Overshoot (%)	*0.0000*	*0.0000*
Undershoot (%)	*0.2404*	*0.78*

The step responses of the two controllers tuned with the penalized cost functions are plotted together in Figure 24 to visually compare their performance in achieving the zero-undershoot objective.

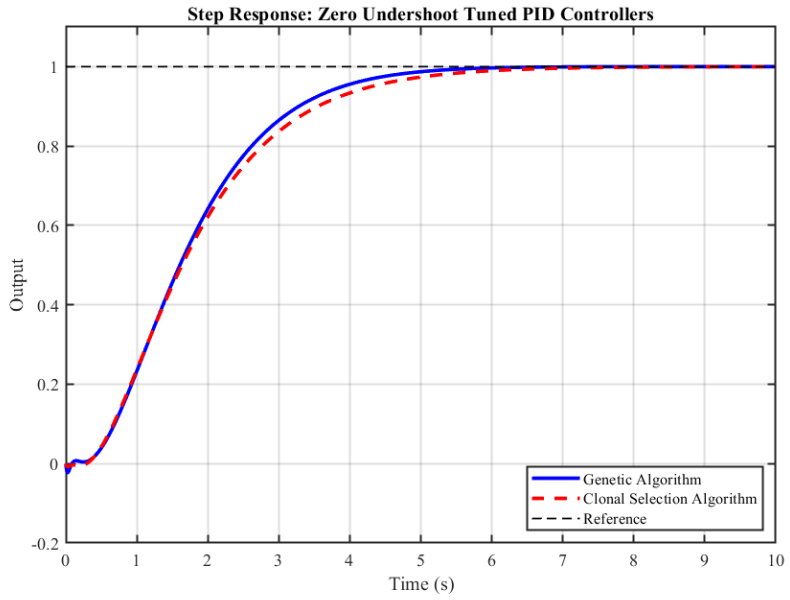


Figure 24: Comparative step response of PID controllers from zero-undershoot tuning methods.

5.3. Discussion

Across all tuning strategies explored for the self balancing robot, the PID controller consistently stood out as the most reliable option in terms of stability, accuracy, and robustness, especially when optimized using metaheuristic algorithms. While the Ziegler Nichols method delivered the fastest response with the P controller, it came at the cost of considerable overshoot and steady state error. The PI controller offered some improvement by reducing overshoot and enhancing steady state accuracy, but the PID controller was ultimately more effective, providing quicker settling with better control, though at the expense of higher undershoot in its standard form.

For both the Genetic Algorithm (GA) and Clonal Selection Algorithm (CSA), incorporating a penalized cost function to reduce undershoot led to noticeably more stable and refined responses. The unpenalized versions of GA and CSA produced faster reactions but introduced moderate overshoot and undershoot, potentially problematic for a physical robot that must avoid excessive oscillation or tipping. On the other hand, when undershoot was explicitly penalized, both algorithms were able to generate PID controllers with zero overshoot and greatly reduced undershoot. Specifically, the GA tuned PID achieved an undershoot of just 0.2779 percent, while the CSA tuned PID achieved 0.78 percent, with both showing clean and smooth responses.

Among all controller types P, PI, and PID, the PID configuration clearly delivered the best overall performance in both standard and penalized tuning scenarios. PI controllers, while stable, became noticeably slower when undershoot was penalized, and P controllers consistently suffered from steady state error, making them unsuitable for this application. MATLAB's Auto Tuner produced acceptable results with both PI and PID controllers, but it lacked the fine tuned control over undershoot that the metaheuristic methods provided.

In summary, penalized PID controllers, especially those tuned using GA and CSA, struck the best balance between speed and stability, making them the most effective option for maintaining balance in a two wheeled self balancing robot. And while both GA and CSA performed closely under undershoot penalized conditions, the Genetic Algorithm held a slight edge, achieving lower undershoot and a slightly faster settling time,

6. Conclusion

This study investigated the performance of classical P, PI, and PID controllers in stabilizing a two wheeled self balancing robot, with a particular focus on how different tuning methods affect system response. The robot's dynamics were modeled using Lagrangian mechanics, linearized around the upright position, and used to evaluate controller performance both analytically and through simulation. Controllers were tuned using the Ziegler Nichols method, MATLAB's built in auto tuner, and two metaheuristic algorithms — Genetic Algorithm (GA) and Clonal Selection Algorithm (CSA). Among the three controller types, the PID controller consistently delivered the most reliable and well rounded performance across all metrics, particularly in minimizing overshoot, improving settling time, and eliminating steady state error.

A key focus of this study was on minimizing undershoot, a crucial performance factor for balance critical systems. While traditional methods like Ziegler Nichols and MATLAB's tuner offered quick or reasonable tuning options, they lacked fine control over undershoot behavior. In contrast, the metaheuristic algorithms allowed for flexible cost function design. When undershoot was penalized in the optimization process, both GA and CSA produced controllers with smooth, stable responses and zero overshoot. Although both approaches performed closely, GA achieved slightly better results, with lower undershoot and a marginally faster settling time.

The key insights and outcomes of this work can be summarized as follows:

(i) PID controllers offered the best overall performance among P, PI, and PID structures, particularly when tuned with intelligent algorithms.

(ii) P controllers consistently exhibited steady state error, making them unsuitable for balance maintenance in self balancing robots.

(iii) Ziegler Nichols tuning produced fast responses but resulted in large overshoot and undershoot, limiting its usefulness in sensitive applications.

(iv) MATLAB's auto tuner provided acceptable PI and PID responses but lacked fine tuning capabilities, especially in managing undershoot.

(v) Metaheuristic tuning using GA and CSA allowed precise control over performance criteria. Penalizing undershoot led to significantly more stable and accurate responses.

(vi) Genetic Algorithm tuning achieved the best result under penalized conditions, with an undershoot of only 0.2779 percent and slightly faster settling compared to CSA, making it the most effective approach in this study.

Acknowledgement

We sincerely thank Professor Dr. Sumon Saha, course teacher of ME461: Control Engineering, for his valuable guidance and support throughout this study.

Data Availability

The data can be obtained from the corresponding author upon a reasonable request.

References

[1] B. Mahler, J. Haase, Balancing an inverted pendulum on a mobile robot, in: 2013 19th International Conference on Automation and Computing (ICAC), IEEE, 2013, pp. 1–6. doi:10.1109/IConAC.2015.7313975.

[2] P. Frankovský, L. Dominik, A. Gmiterko, I. Virgala, P. Kuryło, O. Perminova, Modeling of two-wheeled self-balancing robot driven by dc gearmotors, International Journal of Applied Mechanics and Engineering 22 (3) (2017) 739–747. doi:10.1515/ijame-2017-0046.

[3] H. N. Le, M.-K. Pham, D.-H. Pham, T.-V.-A. Nguyen, Trajectory tracking and stabilization of two-wheeled balancing mobile robot with hierarchical and sliding mode control, International Journal of Dynamics and Control (2024). doi:10.1007/s40435-024-01518-0.

[4] O. Halawa, et al., Mechatronics project: Two-wheeled self-balancing robot, unpublished technical report, Department of Mechatronics Engineering, Helwan University (2023).

[5] P. Kumar, O. N. Mehrotra, J. Mahto, Tuning of pid controller of inverted pendulum using genetic algorithm, International Journal of Research in Engineering and Technology (IJRET) 1 (3) (2012) 359–363.
URL `http://www.ijret.org`

[6] Y. Olmez, G. O. Koca, Z. H. Akpolat, Clonal selection algorithm-based control for two-wheeled self-balancing mobile robot, Simulation Modelling Practice and Theory 118 (2022) 102550. doi:10.1016/j.simpat.2022.102550.

[7] I. Hassanzadeh, S. Mobayen, Controller design for rotary inverted pendulum system using evolutionary algorithms, Mathematical Problems in Engineering 2011 (2011) Article ID 572424. doi:10.1155/2011/572424.

[8] B. Yetayew, D. Tesfaye, A comparative analysis of metaheuristic algorithms tuned super twisting sliding mode control of a self-balancing segway, Engineering Science and Technology, an International JournalAccepted/In press (2024).